

Machine Learning Fundamentals for Deep Learning

A concise explanatory foundation in statistics, learning theory, and classical ML

The central problem of machine learning

Machine learning starts with a simple problem: we observe finite data and want to make useful predictions about observations we have not seen.

Let

$$D = \{(x_i, y_i)\}_{i=1}^n$$

be a dataset of inputs x and targets y . We choose a family of predictors f_θ and a loss function ℓ . Training attempts to find parameters

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

This quantity is the **empirical risk**. What we actually care about is performance on the underlying population:

$$R(\theta) = \mathbb{E}_{(X,Y) \sim P}[\ell(f_\theta(X), Y)].$$

The fundamental problem is therefore not “minimize training loss.” It is:

use finite observations to find a predictor with low expected loss on future observations.

Everything else in machine learning—regularization, validation, cross-validation, model selection, augmentation, and much of statistical learning theory—is concerned with this gap between finite observed data and the unknown population.

Data-generating distributions and the i.i.d. assumption

A statistical model begins with assumptions about how observations arise. The standard supervised-learning assumption is that

$$(X_i, Y_i) \stackrel{\text{i.i.d.}}{\sim} P(X, Y).$$

Independent means one observation does not probabilistically depend on another. Identically distributed means every observation is drawn from the same distribution.

Under these assumptions,

$$\frac{1}{n} \sum_i \ell_i$$

is a useful estimate of

$$\mathbb{E}[\ell].$$

The assumption is often only approximately true. Audio clips from the same song, images of the same person, repeated measurements of one subject, or consecutive time-series observations can be strongly dependent. Randomly splitting such observations can put nearly identical information in both training and test sets.

This is why a test split is not automatically a valid test. Its validity depends on whether it represents the population about which we want to make claims.

Probability: the language of uncertainty

A random variable X describes an uncertain quantity. Its distribution specifies how probability is allocated among possible outcomes.

For a discrete variable,

$$P(X = x) = p(x), \quad \sum_x p(x) = 1.$$

For a continuous variable, a density $p(x)$ satisfies

$$P(a \leq X \leq b) = \int_a^b p(x) dx.$$

The expectation is

$$\mathbb{E}[X] = \sum_x xp(x)$$

or

$$\mathbb{E}[X] = \int xp(x) dx.$$

Variance measures spread:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Covariance measures how two variables vary together:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

Correlation is the normalized covariance:

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}.$$

A crucial distinction is that zero correlation does not generally imply independence. Independence is much stronger.

Conditional probability

$$P(Y | X) = \frac{P(X, Y)}{P(X)}.$$

The joint distribution can be decomposed as

$$P(X, Y) = P(Y | X)P(X).$$

Bayes' rule reverses the conditioning:

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}.$$

The terms have useful interpretations:

$$\underbrace{P(Y)}_{\text{prior}} \underbrace{P(X | Y)}_{\text{likelihood}} \underbrace{P(Y | X)}_{\text{posterior}}.$$

This distinction becomes central in generative models, Bayesian inference, and probabilistic deep learning.

Important probability distributions

The Gaussian distribution

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

has density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

The Bernoulli distribution models a binary outcome:

$$P(Y = y) = p^y(1 - p)^{1-y}.$$

The categorical distribution generalizes this to K classes.

The Laplace distribution has density proportional to

$$\exp\left(-\frac{|x - \mu|}{b}\right),$$

and is associated with absolute-error objectives.

The Poisson distribution is useful for counts:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}.$$

Knowing distributions matters because many ML objectives are maximum-likelihood objectives under an assumed conditional distribution.

Expectation, sampling, and why training works

Suppose

$$Z = \ell(f_\theta(X), Y).$$

The population risk is $\mathbb{E}[Z]$. We cannot normally calculate it because $P(X, Y)$ is unknown. Instead we estimate it with

$$\hat{R}_n = \frac{1}{n} \sum_{i=1}^n Z_i.$$

The law of large numbers says, under appropriate conditions,

$$\hat{R}_n \rightarrow \mathbb{E}[Z]$$

as $n \rightarrow \infty$.

This is one of the fundamental reasons empirical risk minimization is possible.

The central limit theorem additionally gives, under suitable conditions,

$$\sqrt{n}(\hat{R}_n - \mathbb{E}[Z]) \xrightarrow{d} \mathcal{N}(0, \text{Var}(Z)).$$

The important idea is not that every ML problem becomes Gaussian. It is that averages of many weakly dependent observations often have predictable asymptotic behavior.

Statistics: estimating unknown quantities

Suppose data are generated from a distribution indexed by an unknown parameter θ . An estimator $\hat{\theta}$ is a function of the data used to estimate θ .

Bias:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

Variance:

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2].$$

Mean squared error:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Bias}(\hat{\theta})^2 + \text{Var}(\hat{\theta}).$$

This is the original statistical meaning of the bias–variance tradeoff. In ML, “bias” and “variance” are also used to describe prediction behavior across training sets, which is related but not identical.

Maximum likelihood

Given observations D and parameters θ ,

$$L(\theta; D) = p(D \mid \theta)$$

is the likelihood viewed as a function of θ .

Maximum likelihood chooses

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} p(D \mid \theta).$$

Because products can be numerically inconvenient, we usually maximize the log-likelihood:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \sum_i \log p(x_i \mid \theta).$$

Equivalently, we minimize negative log-likelihood.

This is a major bridge between statistics and ML: many familiar losses are negative log-likelihoods.

Decision theory and loss

Prediction alone is not the complete statistical problem. We need a loss function specifying what counts as a bad prediction.

A decision rule $\delta(x)$ maps observations to actions. Its expected loss is

$$R(\delta) = \mathbb{E}[L(Y, \delta(X))].$$

The optimal decision depends on the loss.

For squared error, the optimal prediction is the conditional mean:

$$\delta^*(x) = \mathbb{E}[Y \mid X = x].$$

For absolute error, the optimal prediction is a conditional median.

For a classification problem with 0–1 loss, the Bayes-optimal classifier is

$$\delta^*(x) = \arg \max_y P(Y = y \mid X = x).$$

Thus there is no universally “best prediction.” Optimality is always relative to an objective.

Common ML losses and their statistical meaning

For regression,

$$L_{\text{MSE}} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2.$$

If

$$Y \mid X = x \sim \mathcal{N}(\mu(x), \sigma^2)$$

with fixed variance, maximizing likelihood is equivalent to minimizing squared error.

For absolute error,

$$L_{\text{MAE}} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|,$$

the corresponding likelihood interpretation is Laplace noise.

For binary classification, if

$$Y \mid X = x \sim \text{Bernoulli}(p(x)),$$

negative log-likelihood becomes binary cross-entropy:

$$L = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

For K classes,

$$L = -\frac{1}{n} \sum_i \sum_{k=1}^K y_{ik} \log p_{ik}.$$

Therefore cross-entropy is not an arbitrary neural-network formula: it is the negative log-likelihood for a categorical model.

Entropy, cross-entropy, and KL divergence

Entropy measures uncertainty:

$$H(P) = -\sum_x p(x) \log p(x).$$

Cross-entropy measures the expected coding cost when distribution P is represented using distribution Q :

$$H(P, Q) = -\sum_x p(x) \log q(x).$$

KL divergence is

$$D_{\text{KL}}(P \parallel Q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

They satisfy

$$H(P, Q) = H(P) + D_{\text{KL}}(P \| Q).$$

KL divergence is nonnegative and equals zero when the distributions agree almost everywhere. It is not symmetric:

$$D_{\text{KL}}(P \| Q) \neq D_{\text{KL}}(Q \| P).$$

Mutual information is

$$I(X; Y) = D_{\text{KL}}(P_{XY} \| P_X P_Y).$$

It is zero exactly when X and Y are independent.

Statistical learning: generalization

Training minimizes empirical risk:

$$\hat{R}(f) = \frac{1}{n} \sum_i \ell(f(x_i), y_i).$$

But the desired quantity is population risk:

$$R(f) = \mathbb{E}[\ell(f(X), Y)].$$

Their difference,

$$R(f) - \hat{R}(f),$$

is the generalization gap.

A model can have very low training loss and poor test performance because the training sample does not contain enough information to determine which patterns generalize.

Overfitting

Overfitting means that a learned predictor performs substantially worse on new data than on the data used to fit it.

A common pattern is

$$L_{\text{train}} \downarrow \quad \text{while} \quad L_{\text{validation}} \downarrow \text{ then } \uparrow.$$

Overfitting is not synonymous with “too many parameters.” Modern neural networks can have far more parameters than training examples and still generalize well. Parameter count is one aspect of capacity, not a complete definition of effective complexity.

Underfitting

Underfitting occurs when the learned model cannot adequately represent the relevant relationship. Training and validation errors can both remain high.

The useful conceptual decomposition is:

$$\text{test error} \approx \text{approximation error} + \text{estimation error} + \text{optimization error} + \text{irreducible noise},$$

with the exact decomposition depending on the problem and loss.

Approximation error comes from restricting the hypothesis class. Estimation error comes from finite data. Optimization error comes from failing to find the desired solution. Noise represents uncertainty that cannot be predicted from the available information.

Bias–variance in prediction

For squared-error prediction at a fixed x ,

$$\mathbb{E}_D[(Y - \hat{f}_D(x))^2] = \sigma^2(x) + \text{Bias}[\hat{f}_D(x)]^2 + \text{Var}[\hat{f}_D(x)].$$

Here the expectation is over different possible training datasets D .

A highly constrained model can have high bias and low variance. A highly flexible model can have low bias and high variance. Regularization often shifts this balance.

The classical decomposition is useful, but it should not be treated as a universal explanation of modern deep-network behavior.

Regularization and inductive bias

Regularization makes some solutions preferable to others.

L2-regularized optimization:

$$\min_{\theta} \hat{R}(\theta) + \lambda \|\theta\|_2^2.$$

L1:

$$\min_{\theta} \hat{R}(\theta) + \lambda \|\theta\|_1.$$

L1 tends to encourage sparse coefficients because the geometry of the L1 constraint has corners aligned with coordinate axes.

Other regularizers include dropout, data augmentation, early stopping, noise injection, label smoothing, and explicit constraints.

An important modern idea is **implicit regularization**: even when the loss has no explicit penalty, the optimization algorithm and architecture can preferentially select some solutions over others.

Capacity and learning theory

A hypothesis class $\mathcal{F}$ is the set of functions a learning procedure can represent.

Classical learning theory asks how complex $\mathcal{F}$ can be while still allowing empirical performance to tell us something about population performance.

The VC dimension measures the largest number of points that a class can shatter. Rademacher complexity measures, roughly, how well a function class can correlate with random noise.

These quantities can appear in bounds of the form

$$R(f) \lesssim \hat{R}(f) + \text{complexity term} + \text{confidence term}.$$

The precise bounds depend on assumptions and the chosen complexity measure. Their conceptual purpose is more important: finite data cannot support arbitrary complexity without additional structure.

Modern deep learning complicates this classical picture because highly overparameterized networks can interpolate training data and nevertheless generalize. This motivates study of implicit bias, margin, architecture, data geometry, double descent, and algorithm-dependent complexity.

Train, validation, and test data

The training set estimates model parameters.

The validation set is used to make choices such as architecture, regularization, learning rate, or stopping point.

The test set should be reserved for final evaluation.

Repeatedly checking test performance and changing the model creates *test-set overfitting*. Even without explicitly fitting the test labels, repeated decisions based on test results make the test set part of the optimization process.

Cross-validation estimates performance by repeatedly training on subsets and evaluating on held-out folds. It is useful when data are limited, but the splitting procedure must respect dependence. Grouped or temporal data often require specialized splits.

Data leakage

Leakage occurs when information unavailable at prediction time enters the training process.

Examples include:

- calculating normalization statistics using the full dataset before splitting;
- allowing future information into a time-series feature;
- putting near-duplicate samples in both train and test;
- constructing a feature from the target;
- augmenting an example and placing related versions across different splits.

Leakage can produce excellent validation results with a model that fails in deployment. It is therefore a methodological error, not merely a minor implementation bug.

Classification fundamentals

For binary classification,

$$p = P(Y = 1 \mid X = x).$$

The log-odds are

$$\log \frac{p}{1-p}.$$

Logistic regression assumes

$$\log \frac{p}{1-p} = w^\top x + b,$$

giving

$$p = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The model is linear in log-odds, not necessarily in probability.

For K classes, softmax converts logits z_k into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

A numerically stable implementation subtracts

$$m = \max_j z_j$$

before exponentiation.

Evaluation metrics

For binary classification:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

The F1 score is

$$F_1 = \frac{2PR}{P + R}.$$

Accuracy is

$$\frac{TP + TN}{TP + TN + FP + FN}.$$

Accuracy can be misleading under class imbalance. If only 1% of cases are positive, always predicting the negative class gives 99% accuracy while having zero recall for the positive class.

ROC curves examine true-positive rate against false-positive rate across thresholds. Precision-recall curves are often more informative when positive cases are rare.

For regression, common metrics include

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|,$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2},$$

and

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Metrics must be selected according to the actual decision problem, not because one metric is universally superior.

Calibration and probabilistic predictions

A classifier can rank examples correctly while producing badly calibrated probabilities.

If a group of predictions is assigned probability 0.8, a calibrated model should have approximately 80% positives in that group.

Calibration is therefore different from discrimination. A model can have excellent AUC but poor calibration.

Common tools include reliability diagrams, the Brier score,

$$\text{Brier} = \frac{1}{n} \sum_i (p_i - y_i)^2,$$

and post-hoc temperature scaling.

Classical regression

Linear regression assumes

$$y = X\beta + \varepsilon.$$

Ordinary least squares solves

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

When $X^\top X$ is invertible,

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

The normal-equation expression is mathematically useful, although numerical implementations generally use factorizations such as QR or SVD rather than explicitly computing the inverse.

Multicollinearity means predictors contain strongly overlapping information. It can make coefficient estimates unstable even when predictions remain accurate.

Trees and ensembles

A decision tree recursively partitions the feature space. At each node, it chooses a split that improves an impurity objective.

For classification, Gini impurity is

$$G = 1 - \sum_k p_k^2,$$

while entropy is

$$H = - \sum_k p_k \log p_k.$$

Trees can represent nonlinear interactions without explicit feature engineering but can overfit if allowed to grow excessively.

Random forests reduce variance by averaging many randomized trees.

Boosting instead builds models sequentially, with later models focusing on errors left by earlier ones. Gradient boosting can be interpreted as functional gradient descent on a loss.

Nearest neighbors and the curse of dimensionality

k-nearest neighbors predicts from nearby observations. Its central assumption is that nearby points have similar targets.

The problem is that “nearby” becomes less informative in high dimensions. As dimensionality grows, distances can concentrate and the amount of data required to densely cover a space grows rapidly.

This is the curse of dimensionality.

It motivates feature selection, dimensionality reduction, learned representations, and specialized similarity functions.

Support vector machines and kernels

A linear SVM seeks a separating hyperplane with a large margin. The soft-margin objective is

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

subject to

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The margin encourages separation with a buffer around the decision boundary.

Kernel methods replace explicit feature maps with inner products

$$K(x, x') = \phi(x)^\top \phi(x').$$

The kernel trick allows algorithms to operate in high- or infinite-dimensional feature spaces without explicitly constructing $\phi(x)$.

Clustering

Clustering attempts to organize observations without target labels.

k-means solves

$$\min_{\{c_i\}, \{\mu_k\}} \sum_i \|x_i - \mu_{c_i}\|^2.$$

Its objective assumes that Euclidean distance and cluster centroids are meaningful. It therefore performs poorly when clusters have unsuitable shapes or scales.

Gaussian mixture models instead model

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

Unlike k-means, mixture models produce soft assignments and explicitly model a density.

Dimensionality reduction

PCA finds orthogonal directions explaining maximum variance. For centered data with covariance matrix Σ ,

$$\Sigma v_k = \lambda_k v_k.$$

The directions with the largest eigenvalues explain the largest amount of variance.

PCA can also be obtained through the singular value decomposition

$$X = U \Sigma V^\top.$$

PCA is a linear transformation. It is not equivalent to arbitrary representation learning.

Methods such as t-SNE and UMAP are primarily visualization/manifold tools. A visually separated embedding does not prove that the original data contain objectively separated classes.

Latent variables and EM

A latent-variable model introduces an unobserved variable Z that helps explain observations X :

$$Z \rightarrow X.$$

Gaussian mixture models are an example.

Expectation-maximization alternates between estimating the latent assignments and optimizing parameters:

1. E-step: compute posterior responsibilities $P(Z \mid X, \theta)$;
2. M-step: maximize the expected complete-data log-likelihood.

EM typically converges to a local optimum or stationary point, not necessarily the global optimum.

The idea of latent variables is foundational for autoencoders, variational autoencoders, mixture models, hidden Markov models, and many generative models.

Bayesian inference

Bayesian inference combines prior information with observed data:

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}.$$

The posterior is proportional to

$$p(\theta \mid D) \propto p(D \mid \theta)p(\theta).$$

Maximum a posteriori estimation chooses

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} [\log p(D \mid \theta) + \log p(\theta)].$$

This reveals an important connection: a prior can act like a regularizer. For example, a Gaussian prior on parameters produces an L2-type penalty in the MAP objective.

Bayesian prediction integrates over parameter uncertainty:

$$p(y \mid x, D) = \int p(y \mid x, \theta)p(\theta \mid D) d\theta.$$

Uncertainty

Two broad sources of predictive uncertainty are useful to distinguish.

Aleatoric uncertainty is uncertainty inherent in the observation process. For example, two physically similar inputs may genuinely have different outcomes.

Epistemic uncertainty reflects uncertainty about the model or parameters due to limited information.

A deterministic point predictor does not automatically provide either type of uncertainty. Probabilistic models, ensembles, Bayesian methods, and other techniques can estimate aspects of it.

Optimization

Machine learning training is usually an optimization problem:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t).$$

The gradient points in the direction of greatest local increase. Therefore moving against it locally decreases the objective.

Full-batch gradient descent uses all observations to calculate the gradient. Stochastic gradient descent uses one observation; mini-batch SGD uses a subset.

For a mini-batch B ,

$$g_B = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i.$$

Under random sampling, this is often an approximately unbiased estimator of the full gradient.

The noise in mini-batch gradients is not merely an inconvenience. It can influence which solutions optimization reaches.

Momentum and adaptive optimization

Momentum maintains a running velocity:

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

This can reduce oscillation and accelerate movement along persistent directions.

Adam maintains moving averages of gradients and squared gradients:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \end{aligned}$$

Bias correction compensates for initialization near zero.

AdamW separates weight decay from the adaptive gradient update. This distinction matters because an L2 penalty added to the objective is not mathematically identical to decoupled weight decay under adaptive optimization.

Convexity and non-convexity

A function is convex if

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

for $0 \leq \lambda \leq 1$.

Convex optimization has strong theoretical guarantees: every local minimum is global.

Deep neural networks generally produce non-convex objectives. Their landscapes contain saddle points, flat regions, symmetries, and many equivalent parameterizations.

This does not mean optimization is hopeless. Empirical structure, overparameterization, architecture, and stochastic optimization can make useful solutions accessible.

Feature engineering and preprocessing

Classical ML often depends strongly on how raw observations are represented.

Standardization transforms a feature using

$$z = \frac{x - \mu}{\sigma}.$$

It prevents variables with large numerical scales from dominating distance-based or gradient-based procedures.

Categorical variables can be represented using one-hot encoding. High-cardinality variables may require embeddings or other encodings.

Preprocessing parameters must be estimated only from training data. Otherwise preprocessing itself leaks information from validation or test data.

Deep learning reduces the need for manual feature engineering in many domains because intermediate representations can be learned directly from raw or lightly processed data. It does not eliminate the underlying statistical problem of choosing an appropriate representation.

Class imbalance

Suppose only a small fraction of examples belong to the positive class. Standard empirical risk may be dominated by the majority class.

Possible responses include:

- class-weighted losses;
- resampling;
- threshold adjustment;
- specialized objectives;
- collecting more minority-class data.

The correct choice depends on the decision costs. There is no general rule that oversampling or weighting is always superior.

Distribution shift

Training and deployment distributions may differ.

Covariate shift:

$$P_{\text{train}}(X) \neq P_{\text{test}}(X)$$

while the conditional relationship may remain approximately stable.

Label shift changes

$$P(Y).$$

Concept shift changes

$$P(Y | X).$$

A model can therefore fail even if training optimization and test evaluation were both performed correctly, simply because deployment is outside the distribution represented by the benchmark.

This is one reason robustness and domain adaptation are important extensions of ordinary supervised learning.

Causal versus predictive learning

Prediction asks about

$$P(Y \mid X).$$

Causal inference asks what happens under an intervention, often represented as

$$P(Y \mid do(X = x)).$$

These are generally different.

For example, ice-cream sales and drowning deaths can be correlated because temperature affects both. Observing X does not mean manipulating X will produce the same relationship.

Causal reasoning uses additional structure, such as directed acyclic graphs, interventions, assumptions about confounding, or experimental randomization.

Deep learning can be an excellent predictive tool without being a causal model.

Statistical hypothesis testing

A hypothesis test compares a null hypothesis H_0 against an alternative.

A p-value is

$$P(\text{data at least this extreme} \mid H_0).$$

It is not

$$P(H_0 \mid \text{data}).$$

A small p-value indicates that the observed result would be relatively unusual under the specified null model. It does not measure the size or practical importance of an effect.

Type I error is rejecting a true null. Type II error is failing to reject a false null.

Statistical power is the probability of detecting an effect of a specified size under a specified alternative.

Large datasets can make tiny effects statistically significant, while small datasets can fail to detect important effects. Effect size and uncertainty intervals therefore matter alongside significance.

Multiple comparisons

If many hypotheses are tested, some will appear significant by chance.

If each independent test uses significance level α , the probability of at least one false positive can approach

$$1 - (1 - \alpha)^m$$

for m tests.

Corrections such as Bonferroni control family-wise error:

$$\alpha_{\text{per test}} = \frac{\alpha}{m}.$$

False discovery rate procedures instead control the expected proportion of false discoveries among rejected hypotheses. The same principle appears in ML experimentation: repeatedly trying architectures, preprocessing choices, seeds, and metrics against a fixed validation set creates selection bias.

Experimental design and reproducibility

A reliable ML experiment specifies before evaluation:

- the target population;
- the data split;
- the metric;
- the evaluation protocol;
- important preprocessing;

- the model-selection procedure.

Random seeds do not make an experiment scientifically valid. Reproducibility also depends on data versions, preprocessing, software, hardware, initialization, optimization settings, and evaluation code.

A useful distinction is:

$$\text{reproducibility} \neq \text{generalization}.$$

A perfectly reproducible experiment can still produce a model that does not generalize.

Representation learning and embeddings

A representation transforms input x into an intermediate vector:

$$h = f_{\theta}(x).$$

A useful representation makes relevant information accessible to later computation.

An embedding is simply a learned or constructed vector representation. It is not inherently a neural-network concept; classical dimensionality reduction and matrix factorization also produce vector representations.

Cosine similarity is

$$\cos(\theta) = \frac{x^{\top}y}{\|x\|\|y\|}.$$

Contrastive learning trains representations so that related examples become similar and unrelated examples become dissimilar according to a chosen objective.

The important concept is the learned geometry of the representation space, not merely the fact that the output is a vector.

Inductive bias and invariance

An inductive bias is a preference or assumption that makes some solutions easier to learn than others.

A convolutional model assumes useful structure is local and translation-related. A tree assumes that recursive feature-based partitions can describe the target. A linear model assumes the relevant relationship can be represented linearly in its features.

An invariant representation satisfies, approximately,

$$f(Tx) = f(x)$$

for transformations T that should not change the target.

An equivariant representation satisfies a structured transformation rule:

$$f(Tx) = T'f(x).$$

These ideas explain much of the value of neural architectures: architecture can encode useful assumptions before learning begins.

Dimensionality, distance, and the geometry of data

High-dimensional spaces behave differently from low-dimensional intuition.

As dimension grows, volume becomes concentrated in unusual regions and nearest-neighbor distances can become less discriminative. Data may occupy a much lower-dimensional manifold or structured subset of the ambient space.

This helps explain why learned representations can be powerful: rather than treating every coordinate of the raw input as equally meaningful, a model can transform the data into a space in which relevant relationships are easier to represent.

Generative versus discriminative modeling

A discriminative model directly represents

$$P(Y | X)$$

or a decision function.

A generative model represents the data distribution, for example

$$P(X)$$

or

$$P(X, Y) = P(Y)P(X | Y).$$

Generative modeling enables sampling:

$$x \sim P_\theta(X).$$

Autoregressive models factorize a distribution:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t}).$$

This factorization is fundamental to language models and many sequential generative systems.

The conceptual bridge to deep learning

A neural network can be viewed as a parameterized function

$$f_\theta(x)$$

inside exactly the same statistical framework introduced at the beginning.

Training still attempts to minimize empirical risk:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i \ell(f_\theta(x_i), y_i).$$

What changes is the expressive function class and the way its parameters are optimized.

A multilayer network constructs

$$h_1 = \phi(W_1x + b_1),$$

$$h_2 = \phi(W_2h_1 + b_2),$$

and eventually

$$\hat{y} = f_\theta(x).$$

The classical concepts therefore remain present:

probability	→ output distributions
likelihood	→ training objectives
regularization	→ generalization
optimization	→ backpropagation + SGD
representation	→ hidden layers
inductive bias	→ architecture
statistics	→ evaluation and uncertainty.

The most important conceptual distinctions

Several distinctions prevent major misunderstandings.

Parameter vs. hyperparameter. Parameters are learned from the training objective. Hyperparameters control the learning procedure or model and are selected externally.

Training error vs. generalization error. Training error concerns observed data. Generalization error concerns the population or future data.

Probability vs. likelihood. $P(X | \theta)$ is a probability model viewed as a function of X ; the same expression viewed as a function of θ is a likelihood.

Correlation vs. causation. Statistical association does not identify the result of intervention.

Accuracy vs. calibration. Accuracy concerns decisions; calibration concerns whether predicted probabilities correspond to frequencies.

Model capacity vs. parameter count. Parameter count is an imperfect proxy for effective complexity.

Regularization vs. optimization. Regularization changes which solutions are preferred. Optimization determines which solution is reached.

Prediction vs. explanation. A model can predict well without revealing a causal mechanism.

Unsupervised vs. self-supervised learning. Self-supervised learning creates targets from the data; it is often best understood as supervised optimization with automatically generated labels.

What “understanding ML” means before deep learning

The essential mental model is:

data → probabilistic assumptions → model class → loss → optimization → fitted predictor → evaluation → generalization.

Probability describes uncertainty. Statistics describes how unknown quantities can be learned from finite observations. Machine learning turns those ideas into procedures for prediction and representation. Deep learning changes the function class and optimization scale, but it does not remove the statistical foundations.

The central question remains:

Why should a function fitted to finite data work on data it has never seen?

Understanding generalization, rather than merely knowing algorithms, is the conceptual foundation on which deep learning rests.